SimCT: Recovering Lost Supervision for Cross-Tokenizer On-Policy Distillation

Jie Sun[1,3,*,‡], **Mao Zheng**[2,*], **Mingyang Song**[2,*], **Qiyong Zhong**[1]
Yilin Cheng[4], **Bichuan Feng**[4], **Pengfei Liu**[3], **Junfeng Fang**[1,†], **Xiang Wang**[1,†]

[1] University of Science and Technology of China [2] Large Language Model Department, Tencent
[3] Shanghai Innovation Institute [4] Zhongguancun Academy

sunjie2019@mail.ustc.edu.cn

arXiv:2605.07711v2 [cs.CL] 21 May 2026

Abstract

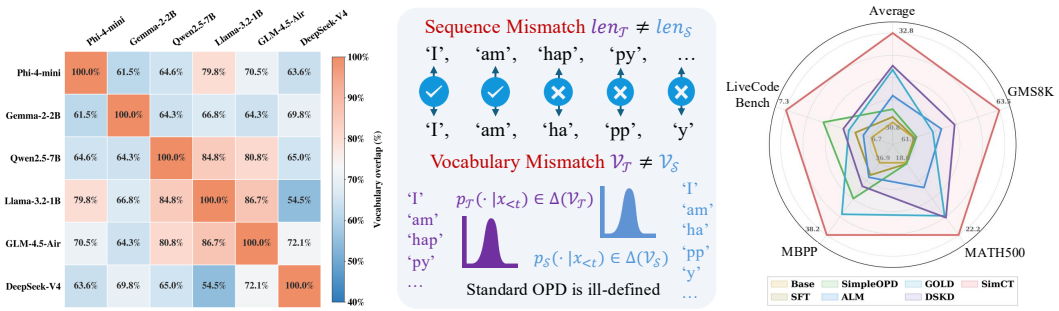
On-policy distillation (OPD) is a standard tool for transferring teacher behavior to a smaller student, but it implicitly assumes that teacher and student predictions are comparable token by token, an assumption that fails whenever the two models tokenize the same text differently. Under heterogeneous tokenizers, exact shared-token matching silently discards a large fraction of the teacher signal at precisely the positions where vocabularies disagree. We propose **Simple Cross-Tokenizer OPD (SimCT)**, which restores this signal by enlarging the supervision space: alongside shared tokens, SimCT compares teacher and student over short multi-token continuations that both tokenizers can realize, leaving the OPD loss form itself unchanged. We show that these units are the finest jointly tokenizable supervision interface, and that coarser alternatives remove teacher-student distinctions that are useful for on-policy learning. Across three heterogeneous teacher-student pairs on mathematical reasoning and code-generation benchmarks, SimCT shows consistent gains over shared-vocabulary OPD and representative cross-tokenizer baselines, with ablations confirming that the improvements come from recovering supervision discarded by exact shared-token matching. Code is available at https://github.com/sunjie279/SimCT-.

1 Introduction

Knowledge distillation transfers the behavior of a strong teacher to a smaller student and remains central to efficient LLM deployment [1, 2, 3, 4]. For autoregressive LLMs, however, conventional offline knowledge distillation trains the student on fixed contexts rather than on states induced by its own generations [5, 6, 7], so early generation errors can compound and move the student into contexts that were rarely supervised during training [8, 9, 10, 11]. On-policy distillation (OPD) addresses this train–test mismatch by querying the teacher on prefixes sampled from the student's policy, following the broader principle of on-policy imitation learning [12, 13, 14, 15, 16]. This makes OPD a natural paradigm for LLM distillation [17, 18, 19, 20, 21], but it also exposes a hidden requirement for cross-model supervision: at each student-generated prefix, teacher and student predictions must be defined over comparable prediction units.

This requirement is non-trivial in realistic LLM distillation, where teacher and student models come from different families and use different tokenizers [22]. The mismatch arises at two levels. First, at the vocabulary level: as shown in Figure 1(A), modern LLM tokenizers share only a limited portion of their vocabularies across model families, so restricting supervision to tokens that appear in both vocabularies leaves most of the teacher's output distribution unused. Second, at the sequence level: the same text may be segmented into different token sequences, so the teacher's next-token distribution at a student-generated prefix may not supervise the same text unit as the student's next-token distribution [23, 24, 25, 26, 27]. As illustrated in Figure 1(B), this sequence mismatch breaks a

[*] Equal Contribution. [†] Corresponding Authors. [‡] Work done during internship at Tencent.



(A) Limited Vocabulary Overlap (B) Tokenizer Mismatch Breaks OPD (C) SimCT Recovers Lost Supervision

Figure 1: **Motivation and performance of SimCT. (A)** LLM tokenizers often share only partial vocabularies across model families, making shared-token distillation restrictive. **(B)** The same text can induce different token boundaries and prediction spaces, so standard token-level OPD is ill-defined. **(C)** SimCT constructs a common aligned supervision space and achieves the best average Pass@1 over SFT and prior cross-tokenizer baselines. Each axis is independently normalized for visualization; absolute results with standard deviations are reported in Table 1.

basic assumption of standard OPD, namely that teacher and student predict over the same text unit at every supervised position.

Existing cross-tokenizer OPD methods fall into three categories. Distribution-alignment methods align heterogeneous logits through optimal transport, between token distributions [23], jointly at token and sequence levels [28], or in on-policy extensions [29], but require transport computation at every supervised position. Representation-transfer methods unify the output spaces through learned cross-model projections [24, 30], at the cost of additional trainable parameters. Token-bypassing methods abstract above the token level via chunk likelihoods [31], BPE structure [27], dynamic vocabulary mapping [32], or byte-level decoders [26], sidestepping token mismatch but supervising at coarser granularity. Across all three categories, teacher and student predictions are made comparable through added machinery, yet a substantial portion of the teacher-student supervision signal remains beyond reach, and what is recovered comes at substantial cost in computation, parameters, or granularity. This raises a natural question: how simple can the supervision interface be while still recovering most of the teacher signal that tokenizer mismatch would otherwise discard?

We propose **Simple Cross-Tokenizer OPD (SimCT)**, a cross-tokenizer OPD method that changes the supervision space rather than the OPD objective. Instead of forcing teacher and student logits to match only on identical tokens, SimCT induces a normalized preference distribution over a common supervision space and applies the standard reverse-KL OPD loss there [15, 33]. The space contains shared tokens and *minimal aligned units* recovered from tokenizer-mismatched text. A minimal aligned unit is the finest shared text continuation that both tokenizers can express, even if they realize it with different token sequences. SimCT requires no additional parameters, no learned projections, and no transport computation, since the supervision space is constructed directly from the two existing tokenizers. It recovers teacher signal that shared-vocabulary OPD discards, while still avoiding regions where the two tokenizers fundamentally disagree. And because the loss form is unchanged, any gain over shared-vocabulary OPD can be attributed to the supervision interface itself rather than to a new objective. As previewed in Figure 1(C), this supervision-space expansion substantially improves over existing cross-tokenizer OPD baselines.

We evaluate SimCT across three model families on mathematical reasoning and code-generation benchmarks. SimCT consistently improves over SFT and representative cross-tokenizer OPD baselines, achieving an average relative gain of **6.0%** over SFT with modest additional computational cost relative to shared-vocabulary OPD. Ablations further explain where the gains come from: increasing recovered tokenizer-mismatched text supervision improves performance, whereas coarsening minimal aligned units removes KL supervision signal and reduces downstream gains. These trends support our theoretical analysis, which characterizes SimCT as the finest boundary-consistent supervision interface jointly expressible by the teacher and student tokenizers, and shows that coarser interfaces can erase useful within-text teacher–student distinctions. Overall, our results identify supervision-space construction as a key design choice in cross-tokenizer OPD, with minimal aligned units offering a simple way to recover teacher supervision otherwise lost under exact token overlap.

2 Related Work

Knowledge distillation and on-policy distillation. Knowledge distillation transfers teacher behavior to a smaller student and has become a standard tool for efficient LLM deployment [1, 3], with early work treating it as offline compression on fixed corpora [5, 6, 7]. Because the student is supervised under a distribution it does not visit at inference [8, 10], on-policy distillation (OPD) instead queries the teacher on student-generated prefixes [34, 13], with subsequent work refining the objective and setting [14, 15, 16, 17, 35, 36, 37]. A complementary line analyzes *when* and *where* OPD provides signal: distributional compatibility governs success or failure at the sequence level [20, 38], while token-level studies show that supervision quality varies sharply across positions [39, 40]. Most of this work assumes a shared token space between teacher and student; extending OPD to heterogeneous tokenization requires additional supervision-interface design, which we discuss next.

Cross-tokenizer knowledge distillation. When teacher and student tokenize the same text differently, prior work bridges the vocabulary gap at one of several interfaces. Optimal-transport methods align heterogeneous logits directly, either between token distributions [23], jointly at token and sequence levels [28], or layer-wise with chain-of-thought augmentation [41]. Dual-space and representation-matching methods instead unify the output spaces themselves through cross-model attention and its variants [24, 42]. Other methods bypass the token level entirely, matching approximate chunk likelihoods [31], doing context-aware sequence matching with dynamic vocabulary mapping [32], or using bytes as a tokenizer-agnostic interface [26], while related directions reuse the teacher loss as supervision [43], extend cross-tokenizer transfer to preference alignment [25], or bridge tokenizer gaps in embedding-model representation space [44].

3 Methodology

3.1 Problem Setup

Let $\mathcal{T}$ and $\mathcal{S}$ denote the teacher and student models, with vocabularies $\mathcal{V}_T$ and $\mathcal{V}_S$. In on-policy distillation, supervision is provided on prefixes sampled from the student policy rather than a fixed offline dataset. At step t, let $x_{<t}$ denote the student-generated prefix. Both models are conditioned on the same text-level context $x_{<t}$, but they predict the next token over their own vocabularies:

$$p_T(\cdot \mid x_{<t}) \in \Delta(\mathcal{V}_T), \qquad p_S(\cdot \mid x_{<t}) \in \Delta(\mathcal{V}_S). \tag{1}$$

Here, $p_T(v \mid x_{<t})$ is the probability the teacher assigns to token $v \in \mathcal{V}_T$, and $p_S(u \mid x_{<t})$ is defined analogously for student token $u \in \mathcal{V}_S$. We write $\Delta(\mathcal{V})$ for the probability simplex over $\mathcal{V}$. When teacher and student share a tokenizer, the two distributions in Eq. (1) are defined over a common prediction space, and standard OPD compares them directly, e.g., by minimizing a token-level KL:

$$\mathcal{L}_{\mathrm{OPD}}(x_{<t}) = \mathrm{KL}(p_S(\cdot \mid x_{<t}) \,\|\, p_T(\cdot \mid x_{<t}))\,. \tag{2}$$

This objective is well defined only because each student token corresponds to the same prediction unit as the corresponding teacher token. Thus, standard OPD implicitly assumes that teacher and student probabilities live on the same simplex and can be compared token by token.

Cross-tokenizer OPD breaks this assumption in two coupled ways. First, the vocabularies differ, $\mathcal{V}_T \neq \mathcal{V}_S$, so the teacher and student distributions in Eq. (1) are supported on different probability spaces. Second, the same text may be segmented into different token sequences, so textually equivalent texts do not necessarily admit a one-to-one token alignment. We refer to these obstacles as *vocabulary mismatch* and *sequence mismatch*, respectively. The goal of cross-tokenizer OPD is to construct a common supervision interface in which teacher feedback remains comparable to the student's next-token prediction on student-generated prefixes.

3.2 Generalized OPD over a Common Supervision Space

Cross-tokenizer OPD requires comparing two next-token distributions that are defined over different vocabularies and may correspond to different text segmentations. We handle this by replacing the shared-vocabulary assumption with a shared set of prediction units, as illustrated in Figure 2(A). Specifically, we introduce a *common supervision space* $\mathcal{U}$, together with scoring maps

$$\Pi_T : \Delta(\mathcal{V}_T) \to \Delta(\mathcal{U}), \qquad \Pi_S : \Delta(\mathcal{V}_S) \to \Delta(\mathcal{U}), \tag{3}$$

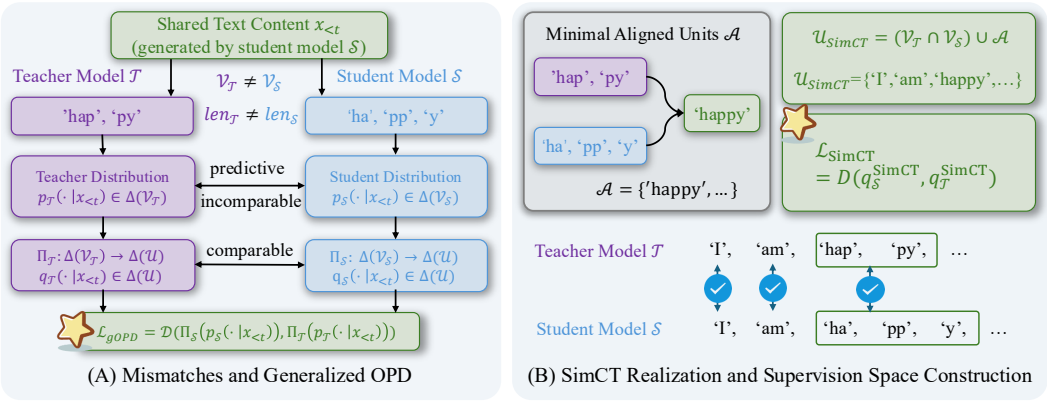


Figure 2: **Framework of SimCT. (A)** In cross-tokenizer OPD, teacher and student next-token distributions are conditioned on the same student-generated prefix, but are defined over different tokenizer spaces. We make them comparable through a common supervision space $\mathcal{U}$. **(B)** SimCT builds $\mathcal{U}_{\text{SimCT}} = (\mathcal{V}_T \cap \mathcal{V}_S) \cup \mathcal{A}$ by adding minimal aligned units $\mathcal{A}$, then applies the original OPD loss on the induced supervision distributions.

which transform teacher and student next-token distributions into comparable distributions over the same prediction units. Given a student-generated prefix $x_{<t}$, these maps induce comparable distributions

$$q_T(\cdot \mid x_{<t}) = \Pi_T(p_T(\cdot \mid x_{<t})), \qquad q_S(\cdot \mid x_{<t}) = \Pi_S(p_S(\cdot \mid x_{<t})). \tag{4}$$

Distillation can then be performed directly in $\mathcal{U}$:

$$\mathcal{L}_{\text{gOPD}}(x_{<t}) = \mathcal{D}(q_S(\cdot \mid x_{<t}), q_T(\cdot \mid x_{<t})), \tag{5}$$

where $\mathcal{D}$ denotes a distillation divergence over the common supervision space.

This formulation recovers standard OPD as a special case. When teacher and student share a tokenizer, we can set $\mathcal{U} = \mathcal{V}_T = \mathcal{V}_S$ and $\Pi_T = \Pi_S = \text{Id}$, reducing Eq. (5) to the token-level objective in Eq. (2). In the cross-tokenizer setting, however, the central design choice is how to construct $\mathcal{U}$ so that it preserves useful teacher supervision while remaining compatible with student predictions. A direct baseline is to use only the shared vocabulary $\mathcal{U}_{\text{shared}} = \mathcal{V}_T \cap \mathcal{V}_S$, and restrict both distributions to this subspace. We refer to this baseline as **SimpleOPD**. SimpleOPD is a natural but conservative instantiation of Eq. (5): it avoids comparing tokens that are not shared by both vocabularies, but it discards supervision signal from texts that are semantically comparable yet represented by different token segments. Thus, SimpleOPD fails not because generalized OPD is inadequate, but because exact token overlap defines an overly narrow supervision space.

3.3 SimCT: Recovering Supervision via Minimal Aligned Units

To expand the common supervision space that goes beyond exact token overlap, we propose **Simple Cross-Tokenizer OPD (SimCT)**. Its central principle is to recover teacher supervision whenever teacher and student predictions refer to the same underlying text, even if the two tokenizers realize that text with different token boundaries. Figure 2(B) gives a simple example: the teacher tokenizes "happy" as "hap", "py", while the student tokenizes the same text as "ha", "pp", "y". Although not comparable as individual tokens, these refer to the same text, so SimCT treats it as a recoverable supervision unit. To construct these units, SimCT considers texts where the two tokenizations disagree and forms the finest boundary-consistent text units realizable by both tokenizers. We call these units *minimal aligned units*, and denote the set of all such units by $\mathcal{A}$. SimCT then defines its supervision space as

$$\mathcal{U}_{\text{SimCT}} = (\mathcal{V}_T \cap \mathcal{V}_S) \cup \mathcal{A}. \tag{6}$$

The shared-vocabulary term keeps supervision that is already comparable on 1:1-aligned shared tokens, while $\mathcal{A}$ recovers supervision from texts that tokenize differently but are comparable after mapping to common aligned units.

We now define the SimCT scoring maps Π_M^{SimCT} for $M \in \{\mathcal{T}, \mathcal{S}\}$. SimCT first assigns each unit $u \in \mathcal{U}_{\mathrm{SimCT}}$ a continuation score and then normalizes over the candidate supervision set. For a 1:1-aligned shared token $u \in \mathcal{V}_T \cap \mathcal{V}_S$, we use the original next-token log-probability, $s_M(u \mid x_{<t}) = \log p_M(u \mid x_{<t})$. For a minimal aligned unit $u \in \mathcal{A}$, let $\tau_M(u) = (v_1, \ldots, v_k)$ be its tokenizer-M realization, and define

$$s_M(u \mid x_{<t}) = \frac{1}{k} \log p_M(u \mid x_{<t}) = \frac{1}{k} \sum_{j=1}^{k} \log p_M(v_j \mid x_{<t}, v_{<j}). \tag{7}$$

The factor $1/k$ uses the average log-likelihood rather than the raw sequence log-likelihood, avoiding systematically penalizing units that require more tokens under a given tokenizer [45]. This normalization is a standard practical choice, not a requirement of the theoretical construction in §3.4. The normalized supervision distribution over the candidate set is then

$$q_M^{\mathrm{SimCT}}(u \mid x_{<t}) = \frac{\exp(s_M(u \mid x_{<t}))}{\sum_{u' \in \mathcal{U}_{\mathrm{SimCT}}} \exp(s_M(u' \mid x_{<t}))}, \tag{8}$$

which we denote as $q_M^{\mathrm{SimCT}} = \Pi_M^{\mathrm{SimCT}}(p_M)$. Importantly, q_M^{SimCT} should not be interpreted as a mass-preserving marginalization of the model's next-token distribution. For multi-token units, SimCT uses length-normalized continuation likelihoods as scores and normalizes them over the candidate supervision set. The resulting distribution is thus an operational scoring interface for cross-tokenizer supervision, rather than a generative probability projection over a complete partition of all possible continuations.

The SimCT loss is the generalized OPD loss instantiated on this recovered supervision space:

$$\mathcal{L}_{\mathrm{SimCT}}(x_{<t}) = \mathcal{D}\big(q_S^{\mathrm{SimCT}}(\cdot \mid x_{<t}), q_T^{\mathrm{SimCT}}(\cdot \mid x_{<t})\big)\,. \tag{9}$$

SimCT performs OPD over the next continuation units rather than tokenizer-specific next tokens. For a multi-token unit, Eq. (7) scores the candidate continuation under the current prefix $x_{<t}$ through its autoregressive factors $p_M(v_j \mid x_{<t}, v_{<j})$, without using future ground-truth tokens or future rollout states. Thus, SimCT preserves the on-policy training form while changing the supervision interface from exact token overlap to aligned continuation units. The procedure is summarized in Algorithm 1.

3.4 Properties of the SimCT Supervision Space

We present two theoretical results that clarify the design of the SimCT supervision space. First, minimal aligned units provide the finest comparable text units jointly expressible by the teacher and student tokenizers. Second, merging these units into coarser supervision units can remove teacher–student distinctions that are visible at the minimal-unit level. Formal definitions, assumptions, and proofs are provided in Appendix D.

Theorem 1 (Minimal aligned units)**.** *For any tokenizer-mismatched text region, SimCT constructs a partition into minimal aligned units. Each unit is jointly tokenizable: it can be represented by one or more teacher tokens and also by one or more student tokens. Moreover, no unit admits a strictly finer non-empty decomposition that preserves joint tokenizability.*

Theorem 1 shows that SimCT constructs the finest boundary-consistent interface on which teacher and student predictions can be compared, making the supervision space more expressive than exact shared-token matching while avoiding unconstrained character-level comparison.

Proposition 1 (Signal loss under coarser supervision spaces)**.** *For a fixed student-generated prefix, let $q_T^{\min}$ and $q_S^{\min}$ be teacher and student distributions over SimCT minimal aligned units. Let $q_T^{\mathcal{C}}$ and $q_S^{\mathcal{C}}$ be the distributions obtained by a non-trivial coarsening $\mathcal{C}$, which sums probabilities inside each coarse unit. Then*

$$\mathrm{KL}\big(q_S^{\min} \,\|\, q_T^{\min}\big) \geq \mathrm{KL}\big(q_S^{\mathcal{C}} \,\|\, q_T^{\mathcal{C}}\big)\,. \tag{10}$$

The gap is exactly the within-unit teacher–student discrepancy removed by the coarsening.

Proposition 1 motivates SimCT to preserve minimal aligned units rather than coarser units. In its idealized distributional setting, coarsening removes the within-unit teacher–student discrepancy that provides a fine-grained distillation signal; Section 4.4 measures this erased signal by the KL gap over student-generated prefixes. Together with Theorem 1, this supports using the finest tokenizer-compatible supervision units. The practical SimCT objective in Eq. (8) implements this principle with normalized continuation scores over a finite candidate set, not as an exact mass-preserving marginal.

4 Experiments

4.1 Experimental Setup

Models and pairs. We construct three teacher–student pairs that span all three model families and tokenizers: Qwen2.5-7B-Instruct [46] → Phi-4-mini-Instruct [47], Qwen → Gemma-2-2B-IT [48], and Phi → Gemma. All pairs follow the realistic large-to-small distillation direction. Two distinct models (Qwen and Phi) serve as teachers and two distinct models (Phi and Gemma) serve as students across pairs, so our results do not hinge on any single teacher or student choice.

Data and warm-start. We use a single teacher-generated corpus to warm-start every student before OPD. Prompts are drawn from GSM8K [49], Orca-Math [50], OpenMathInstruct-1 [51], and MATH excluding MATH-500 [52] for math, and OpenCodeInstruct [53], KodCode [54], TACO [55], and CodeContests [56] for code. For each prompt, the teacher samples 8 responses (temperature 0.6, top-$p = 0.95$); we keep one per prompt via task-specific correctness checks where available and quality filters otherwise, yielding a 10K-example SFT corpus. Each student is then SFT-trained on this corpus to produce the warm-start checkpoint from which all OPD methods start. Full composition, filtering, and license details are in Appendix A.

On-policy distillation. Starting from the warm-start checkpoint, all methods follow an identical OPD loop and differ only in supervision construction: at each step the student samples a prefix under its current policy, the teacher is queried on this prefix to produce next-token supervision, and the student is updated under each method's loss. Training prompts, prefix-generation settings, optimizer, schedule, and compute budget are held identical across methods. Hyperparameters are in Appendix B.

Baselines. We compare SimCT against four baselines that span the main families of cross-tokenizer distillation. SimpleOPD performs exact shared-vocabulary OPD, supervising only the tokens that appear in both vocabularies. DSKD [24] unifies teacher and student output spaces through a learned cross-model projection, requiring additional trainable parameters. ALM [31] matches approximate likelihoods over tokenizer-induced chunks, providing supervision at the chunk level rather than the token level. GOLD [29] extends universal logit distillation [23] to on-policy training with text-aligned probability merging across tokenizer boundaries.

Benchmarks. We evaluate mathematical reasoning on GSM8K [49] and MATH-500 [57], and code generation on MBPP [58] and LiveCodeBench-v6 [59]. We report pass@1 averaged over 5 runs under zero-shot prompting; full templates, decoding settings, and scoring details are in Appendix C.

4.2 Overall Effectiveness

Table 1 reports the main results under the cross-tokenizer OPD setting. All distillation methods start from the same SFT-warm-started student checkpoint and follow the same training protocol. As reported in Appendix E, SimCT only slightly increases the wall-clock cost over SimpleOPD and remains more efficient than baselines. From these results, we draw the following observations:

- **Obs 1: SimCT improves cross-tokenizer OPD consistently across teachers, students, and task domains.** SimCT achieves the best average performance for all three teacher–student pairs and obtains the strongest result on every evaluated benchmark. The gains hold when Qwen2.5-7B-Instruct distills to two different student families, and also when Phi-4-mini-instruct serves as the teacher for Gemma-2-2B-it. They also appear on both mathematical reasoning and code-generation benchmarks. This consistency suggests that SimCT is not tied to a particular teacher, student, or task type, but provides a robust supervision interface for cross-tokenizer OPD.
- **Obs 2: Exact shared-token supervision is too conservative under tokenizer mismatch.** SimpleOPD uses the most direct common supervision space: the exact shared vocabulary. However, it improves only marginally over the SFT warm start across the three distillation settings. This indicates that shared tokens preserve only a limited portion of useful teacher feedback when teacher and student tokenizers differ. By adding minimal aligned units, SimCT recovers supervision from text continuations that are comparable but not represented as identical tokens, leading to consistently stronger performance than SimpleOPD.

Table 1: Main results in the cross-tokenizer OPD setting. All distillation methods start from the same SFT-warm-started student checkpoint for each teacher–student pair and follow the same training protocol. We report mean pass@1 over 5 evaluation runs, with standard deviations shown for each benchmark column. The Average column is computed over the four benchmark means. Best and second-best student results within each teacher–student pair are shown in **bold** and underlined.

Model	Method	Math		Code		Average
		GSM8K	MATH500	MBPP	LCB-v6	
Gemma-2-2B-it	Base	61.85 ±0.25	17.96 ±0.38	36.92 ±0.30	6.65 ±0.23	30.83
Phi-4-mini-instruct	Base	85.75 ±0.31	54.58 ±0.49	55.36 ±0.39	7.92 ±0.30	50.90
Qwen2.5-7B-Instruct	Base	91.43 ±0.33	66.20 ±0.54	65.80 ±0.43	13.71 ±0.37	59.28
Qwen2.5-7B-Instruct teacher ↓ Gemma-2-2B-it student	SFT	61.86 ±0.22	18.02 ±0.35	37.14 ±0.28	6.77 ±0.20	30.95
	SimpleOPD	61.90 ±0.25	18.02 ±0.40	37.56 ±0.31	7.01 ±0.22	31.12
	ALM	62.39 ±0.27	19.41 ±0.44	37.18 ±0.30	6.71 ±0.21	31.42
	GOLD	62.22 ±0.30	21.06 ±0.48	37.84 ±0.34	6.82 ±0.23	31.99
	DSKD	62.65 ±0.31	21.17 ±0.49	37.34 ±0.32	6.86 ±0.24	32.01
	SimCT	**63.53** ±0.42	**22.18** ±0.52	**38.21** ±0.35	**7.29** ±0.25	**32.80**
Qwen2.5-7B-Instruct teacher ↓ Phi-4-mini-instruct student	SFT	86.06 ±0.28	55.16 ±0.45	55.40 ±0.36	8.57 ±0.28	51.30
	SimpleOPD	87.04 ±0.30	56.82 ±0.50	55.02 ±0.38	9.23 ±0.31	52.03
	ALM	87.81 ±0.33	59.84 ±0.56	55.12 ±0.40	10.86 ±0.37	53.41
	GOLD	88.32 ±0.34	60.14 ±0.59	55.08 ±0.41	12.06 ±0.42	53.90
	DSKD	88.63 ±0.34	59.68 ±0.55	55.78 ±0.40	11.43 ±0.39	53.88
	SimCT	**90.22** ±0.40	**61.42** ±0.60	**56.04** ±0.41	**12.57** ±0.44	**55.06**
Phi-4-mini-instruct teacher ↓ Gemma-2-2B-it student	SFT	62.32 ±0.22	18.22 ±0.36	37.48 ±0.29	6.43 ±0.20	31.11
	SimpleOPD	62.62 ±0.25	19.02 ±0.42	37.89 ±0.32	6.43 ±0.22	31.49
	ALM	62.24 ±0.25	19.80 ±0.45	37.80 ±0.33	6.63 ±0.23	31.62
	GOLD	62.52 ±0.26	20.36 ±0.47	38.82 ±0.35	6.57 ±0.24	32.07
	DSKD	62.37 ±0.26	19.80 ±0.44	38.64 ±0.34	6.57 ±0.23	31.85
	SimCT	**62.92** ±0.31	**21.10** ±0.49	**39.16** ±0.36	**7.14** ±0.27	**32.58**

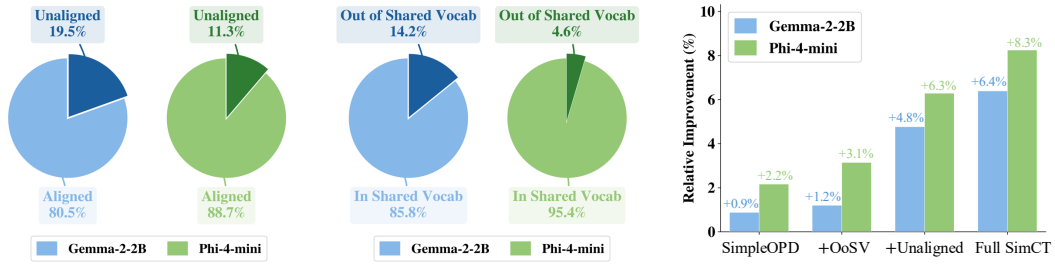
- **Obs 3: Prior cross-tokenizer interfaces are useful but less reliable across settings.** ALM, GOLD, and DSKD often improve over SimpleOPD, confirming that cross-tokenizer supervision beyond exact token overlap is beneficial. However, their relative rankings vary across teacher–student pairs and benchmarks: no existing baseline gives a uniformly strongest interface. This suggests that making heterogeneous predictions comparable is necessary but not sufficient for cross-tokenizer OPD. The supervision interface must also preserve local teacher–student distinctions needed for on-policy next-token feedback, which SimCT addresses through minimal aligned units.

Overall, these observations support the central design choice of SimCT: keep the OPD training paradigm unchanged, but improve the supervision space in which teacher feedback is expressed. Minimal aligned units provide a more robust supervision interface than exact shared tokens or prior cross-tokenizer mappings, leading to consistent gains in realistic cross-tokenizer OPD settings. Appendix F presents a case study with math and code examples showing how different supervision interfaces affect local reasoning or implementation errors.

4.3 Supervision Recovery under Tokenizer Mismatch

We next ask whether SimCT improves cross-tokenizer OPD by recovering teacher supervision that is discarded under tokenizer mismatch. Figure 3 decomposes the missing supervision, while Figure 4 tests whether recovering this supervision leads to downstream gains. From these analyses, we draw the following observations:

- **Obs 4: Tokenizer mismatch removes useful teacher feedback through both sequence mismatch and vocabulary mismatch.** Figure 3(A) shows that a non-trivial fraction of teacher- and student-side tokens cannot be aligned one-to-one. This indicates that standard token-level matching fails to use teacher feedback on units that correspond to the same underlying text but are realized with different tokenizer boundaries. Figure 3(B) reveals a complementary failure mode: even at token-to-token-aligned positions, many high-probability teacher predictions fall outside the shared



(A) Token Alignment Ratio (B) Top-100 Shared Vocab Coverage (C) Supervision Recovery Decomposition

Figure 3: **Supervision recovery under tokenizer mismatch. (A)** Many teacher and student tokens are not aligned one-to-one, revealing supervision loss from sequence mismatch. **(B)** Even at aligned positions, high-probability teacher predictions may fall outside the shared vocabulary. **(C)** Recovering either missing source improves over Base, while Full SimCT performs best by recovering both in a common aligned-unit space. **+Out of Shared Vocab (+OoSV)** adds non-shared vocabulary supervision at aligned positions; **+Unaligned** adds supervision from non-1:1 aligned units.

vocabulary. Together, these results show that SimpleOPD is conservative in two distinct ways: it discards supervision from non-1:1 aligned texts, and it also removes non-shared vocabulary predictions even when the current positions are aligned.

- **Obs 5: The discarded signals are beneficial rather than incidental tokenizer artifacts.** Figure 3(C) directly tests whether the missing signals identified above improve learning. Starting from SimpleOPD, **+Out of Shared Vocab** adds non-shared vocabulary supervision at token-to-token-aligned positions, while **+Unaligned** adds supervision from tokenizer-mismatched units that cannot be aligned one-to-one. Both variants improve over SimpleOPD for both student models, showing that the discarded supervision signal contains useful distillation information. Full SimCT achieves the largest improvement because it recovers both sources through a unified aligned-unit supervision space.
- **Obs 6: Recovering more mismatch unit supervision consistently improves downstream performance.** Figure 4 (Left) further varies the fraction of tokenizer-mismatched unit supervision incorporated into training. For both Gemma-2-2B and Phi-4-mini, the tendency to improve more recovered supervision is used. This trend strengthens the mechanism suggested by Figure 3: mismatch unit supervision is not merely recoverable, but provides a stable learning signal when mapped into a common supervision space. SimCT exploits this signal by aligning tokenizer-specific continuations into minimal aligned units, rather than discarding them under exact token overlap.

These findings explain why SimpleOPD yields only limited gains in Table 1. Shared-vocabulary distillation preserves only the subset of teacher feedback that is already comparable in the original token spaces. SimCT expands the supervision space to reincorporate supervision signal excluded by shared-vocabulary matching, thereby recovering useful teacher supervision while keeping the original OPD objective unchanged.

4.4 Ablation on Supervision-Space Construction

The previous analysis shows that tokenizer-mismatched units contain useful teacher supervision. We now ask whether the *granularity* of the recovered supervision also matters. Starting from the SimCT supervision space, we construct coarsened variants by merging adjacent minimal aligned units into larger units. Let $q_T^{\min}$ and $q_S^{\min}$ denote the teacher and student distributions over minimal aligned units. For a coarsening $\mathcal{C}$, the corresponding coarse distributions aggregate the probability mass of all minimal units inside each coarse unit:

$$q_T^{\mathcal{C}}(c \mid h) = \sum_{u \in c} q_T^{\min}(u \mid h), \qquad q_S^{\mathcal{C}}(c \mid h) = \sum_{u \in c} q_S^{\min}(u \mid h), \tag{11}$$

where h is the student-generated prefix. This aggregation preserves the total probability assigned to each coarse unit, but collapses the relative teacher–student preferences among its constituent minimal units. We quantify the erased supervision signal as:

$$\Delta_{\mathcal{C}} = \mathbb{E}_h \left[\mathrm{KL}\big(q_S^{\min}(\cdot \mid h) \,\|\, q_T^{\min}(\cdot \mid h)\big) - \mathrm{KL}\big(q_S^{\mathcal{C}}(\cdot \mid h) \,\|\, q_T^{\mathcal{C}}(\cdot \mid h)\big) \right]. \tag{12}$$

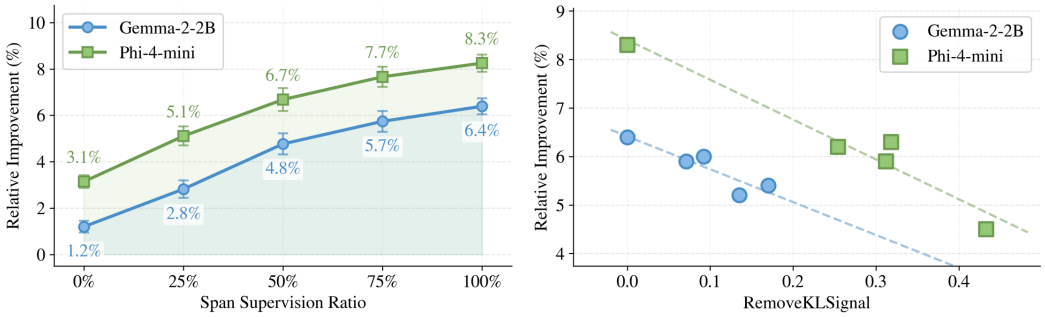


Figure 4: **Ablation on recovered unit supervision and aligned-unit coarsening. (Left)** Recovering more mismatch-unit supervision consistently improves both students. **(Right)** Coarsening minimal aligned units removes within-span KL signal and reduces downstream gains, supporting the need for SimCT's minimal aligned units.

Thus, Δ_C measures the within-unit KL signal removed when minimal aligned units are merged.

- **Obs 7: Coarsening removes useful local supervision.** Figure 4 (Right) shows a negative relationship between removed KL signal and downstream improvement: coarsenings with larger Δ_C yield weaker gains for both Gemma-2-2B-it and Phi-4-mini. This indicates that coarsening does more than simplify the supervision space; it erases local teacher–student preferences useful for OPD. The result supports Proposition 1 and explains why SimCT keeps minimal aligned units instead of merging them into coarser units.

Overall, this ablation shows that SimCT benefits not only from recovering supervision under tokenizer mismatch, but also from preserving it at the finest granularity jointly expressible by both tokenizers.

5 Limitations

Our evaluation centers on math and code reasoning with three teacher–student pairs spanning two teacher families and heterogeneous tokenizers; extending the study to additional domains, scales, and tokenizer families is left to future work. Methodologically, SimCT relies on mapping tokenizer mismatches to minimal aligned units, which is natural for standard BPE-style tokenizers and may require adaptation for tokenizers with substantially different segmentation behavior. Additionally, we adopt $1/k$ (average log-likelihood) normalization as a standard length-correction heuristic, but do not exhaustively study alternatives such as no normalization, or character/byte-level normalization; a systematic calibration study of the induced candidate-space distribution is left to future work. Finally, we target the white-box OPD setting in which teacher next-token distributions are available; extending supervision-space construction to black-box teachers that expose only samples or scalar feedback is a promising direction.

6 Conclusion

We studied cross-tokenizer OPD, where teacher and student policies follow OPD but use different tokenizers. We argued that cross-tokenizer OPD should be viewed as a supervision-space construction problem: useful teacher feedback is lost when heterogeneous token-level predictions are forced to match only on exact shared tokens. We introduced SimpleOPD as a conservative shared-vocabulary baseline and proposed SimCT, which expands the common supervision space with minimal aligned units while preserving the standard OPD objective. Our theoretical analysis characterizes these units as the finest jointly expressible supervision interface and shows that coarsening them can erase useful teacher–student distinctions. Across model families and math/code benchmarks, SimCT consistently improves over SimpleOPD and representative cross-tokenizer distillation baselines. These results suggest that changing the supervision space, rather than the OPD training paradigm, is an effective path for cross-tokenizer OPD.

References

[1] Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the knowledge in a neural network. *arXiv preprint*, arXiv:1503.02531, 2015.

[2] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics (TACL)*, 2024.

[3] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models. *arXiv preprint*, arXiv:2402.13116, 2024.

[4] Chuanpeng Yang, Wang Lu, Yao Zhu, Yidong Wang, Qian Chena, Chenlong Gao, Bingjie Yan, and Yiqiang Chen. Survey on knowledge distillation for large language models: Methods, evaluation, and application. *arXiv preprint*, arXiv:2407.01885, 2024.

[5] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. In *Proceedings of the 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (NeurIPS 2019)*, Vancouver, Canada, 2019.

[6] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174, Online Event, 2020.

[7] Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. Compact language models via pruning and knowledge distillation. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.

[8] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, 2015.

[9] Wen Zhang, Yang Feng, Fandong Meng, Di You, and Qun Liu. Bridging the gap between training and inference for neural machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019)*, pages 4334–4343, Florence, Italy, 2019.

[10] Alexander Lin, Jeremy Wohlwend, Howard Chen, and Tao Lei. Autoregressive knowledge distillation through imitation learning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 6121–6133, Online Event, 2020.

[11] Chris Cundy and Stefano Ermon. SequenceMatch: Imitation learning for autoregressive sequence modelling with backtracking. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, Vienna, Austria, 2024.

[12] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS 2011)*, pages 627–635, Fort Lauderdale, FL, USA, 2011.

[13] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, Vienna, Austria, 2024.

[14] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: On-policy distillation of large language models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, Vienna, Austria, 2024.

[15] Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. DistiLLM: Towards streamlined distillation for large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235, Vienna, Austria, 2024.

[16] Jongwoo Ko, Tianyi Chen, Sungnyun Kim, Tianyu Ding, Luming Liang, Ilya Zharkov, and Se-Young Yun. DistiLLM-2: A contrastive approach boosts the distillation of LLMs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, 2025.

[17] Tianzhu Ye, Li Dong, Zewen Chi, Xun Wu, Shaohan Huang, and Furu Wei. Black-box on-policy distillation of large language models. *arXiv preprint*, arXiv:2511.10643, 2025.

[18] Yecheng Wu, Song Han, and Hai Cai. Lightning OPD: Efficient post-training for large reasoning models with offline on-policy distillation. *arXiv preprint*, arXiv:2604.13010, 2026.

[19] Ijun Jang, Jewon Yeom, Juan Yeo, Hyunggu Lim, and Taesup Kim. Stable on-policy distillation through adaptive target reformulation. *arXiv preprint*, arXiv:2601.07155, 2026.

[20] Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan ang Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint*, arXiv:2604.13016, 2026.

[21] Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint*, arXiv:2602.12275, 2026.

[22] Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint*, arXiv:2604.00626, 2026.

[23] Nicolas Boizard, Kevin El Haddad, Céline Hudelot, and Pierre Colombo. Towards cross-tokenizer distillation: The universal logit distillation loss for LLMs. *Transactions on Machine Learning Research (TMLR)*, 2025.

[24] Songming Zhang, Xue Zhang, Zengkui Sun, Yufeng Chen, and Jinan Xu. Dual-space knowledge distillation for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, pages 18164–18181, Miami, Florida, USA, 2024.

[25] Truong Nguyen, Phi Van Dat, Ngan Nguyen, Linh Ngo Van, Trung Le, and Thanh Hong Nguyen. CTPD: Cross-tokenizer preference distillation. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence (AAAI 2026)*, Philadelphia, PA, USA, 2026.

[26] Avyav Kumar Singh, Yen-Chen Wu, Alexandru Cioba, Alberto Bernacchia, and Davide Buffelli. Cross-tokenizer LLM distillation through a byte-level interface. *arXiv preprint*, arXiv:2604.07466, 2026.

[27] Buu Phan, Ashish Khisti, and Karen Ullrich. Cross-tokenizer likelihood scoring algorithms for language model distillation. In *Proceedings of the 14th International Conference on Learning Representations (ICLR 2026)*, Rio de Janeiro, Brazil, 2026.

[28] Xiao Cui, Mo Zhu, Yulei Qin, Liang Xie, Wengang Zhou, and Houqiang Li. Multi-level optimal transport for universal cross-tokenizer knowledge distillation on language models. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence (AAAI 2025)*, Philadelphia, PA, 2025.

[29] César Miguel Patiño, Kashif Rasul, Quentin Gallouédec, Ben Burtenshaw, Sergio Paniego, Vaishakh Srivastav, Thomas Frere, Edward Beeching, Lewis Tunstall, Leandro von Werra, and Thomas Wolf. Unlocking on-policy distillation for any model family. *Hugging Face Technical Report*, 2025. https://huggingfaceh4-on-policy-distillation.hf.space/unlocking-on-policy-distillation-for-any-model-family.pdf.

[30] Xue Zhang, Songming Zhang, Yunlong Liang, Fandong Meng, Yufeng Chen, Jinan Xu, and Jie Zhou. A dual-space framework for general knowledge distillation of large language models. *arXiv preprint*, arXiv:2504.11426, 2025.

[31] Benjamin Minixhofer, Ivan Vulić, and Edoardo Maria Ponti. Universal cross-tokenizer distillation via approximate likelihood matching. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

[32] Yijie Chen, Yijin Liu, Fandong Meng, Yufeng Chen, Jinan Xu, and Jie Zhou. Enhancing cross-tokenizer knowledge distillation with contextual dynamical mapping. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 8005–8018, Vienna, Austria, 2025.

[33] Woogyeol Jin, Taywon Min, Yongjin Yang, Swanand Ravindra Kadhe, Yi Zhou, Dennis Wei, Nathalie Baracaldo, and Kimin Lee. Entropy-aware on-policy distillation of language models. *arXiv preprint*, arXiv:2603.07079, 2026.

[34] Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20251026. https://thinkingmachines.ai/blog/on-policy-distillation.

[35] Taiqiang Wu, Chaofan Tao, Jiahao Wang, Zhe Zhao, and Ngai Wong. Rethinking kullback-leibler divergence in knowledge distillation for large language models. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, 2025.

[36] Hoang-Chau Luong, Dat Ba Tran, and Lingwei Chen. Diversity-aware reverse kullback-leibler divergence for large language model distillation. *arXiv preprint*, arXiv:2604.00223, 2026.

[37] Yeongmin Kim, Donghyeok Shin, Mina Kang, Byeonghu Na, and Il-Chul Moon. Distillation of large language models via concrete score matching. *arXiv preprint*, arXiv:2509.25837, 2025.

[38] Yuqian Fu, Haohuan Huang, Kaiwen Jiang, Yuanheng Zhu, and Dongbin Zhao. Revisiting on-policy distillation: Empirical failure modes and simple fixes. *arXiv preprint*, arXiv:2603.25562, 2026.

[39] Haiduo Huang, Jiangcheng Song, Yadong Zhang, and Pengju Ren. SelecTKD: Selective token-weighted knowledge distillation for LLMs. *arXiv preprint*, arXiv:2510.24021, 2025.

[40] Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, Zhipeng Wang, and Alborz Geramifard. TIP: Token importance in on-policy distillation. *arXiv preprint*, arXiv:2604.14084, 2026.

[41] Anh Duc Le, Tu Vu, Nam Le Hai, Nguyen Thi Ngoc Diep, Linh Ngo Van, Trung Le, and Thien Huu Nguyen. CoT2Align: Cross-chain of thought distillation via optimal transport alignment for language models with different tokenizers. *arXiv preprint*, arXiv:2502.16806, 2025.

[42] Stella Eva Tsiapali, Cong-Thanh Do, and Kate Knill. Dual-space knowledge distillation with key-query matching for large language models with vocabulary mismatch. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2026)*, 2026.

[43] Haebin Shin, Lei Ji, Xiao Liu, and Yeyun Gong. Overcoming vocabulary mismatch: Vocabulary-agnostic teacher guided language modeling. *arXiv preprint*, arXiv:2503.19123, 2025.

[44] Hai An Vu, Minh-Phuc Truong, Tu Vu, and Linh Ngo Van. MoL: Mixture of layers in cross-tokenizer embedding model distillation. *Knowledge-Based Systems*, 2026.

[45] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint*, arXiv:1609.08144, 2016.

[46] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint*, arXiv:2412.15115, 2024.

[47] Microsoft: Abdelrahman Abouelenin, Atabak Ashfaq, Adam Atkinson, Hany Awadalla, Nguyen Bach, Jianmin Bao, Alon Benhaim, Martin Cai, Vishrav Chaudhary, Congcong Chen, Dong Chen, Dongdong Chen, Jun-Kun Chen, Weizhu Chen, Yen-Chun Chen, Yi-ling Chen, Qi Dai, Xiyang Dai, Ruchao Fan, Mei Gao, Min Gao, Amit Garg, Abhishek Goswami, Junheng Hao, Amr Hendy, Yuxuan Hu, Xin Jin, Mahmoud Khademi, Dongwoo Kim, Young Jin Kim, Gina Lee, Jinyu Li, Yunsheng Li, Chen Liang, Xihui Lin, Zeqi Lin, Mengchen Liu, Yang Liu, Gilsinia Lopez, Chong Luo, Piyush Madan, Vadim Mazalov, Arindam Mitra, Ali Mousavi, Anh Nguyen, Jing Pan, Daniel Perez-Becker, Jacob Platin, Thomas Portet, Kai Qiu, Bo Ren, Liliang Ren, Sambuddha Roy, Ning Shang, Yelong Shen, Saksham Singhal, Subhojit Som, Xia Song, Tetyana Sych, Praneetha Vaddamanu, Shuohang Wang, Yiming Wang, Zhenghao Wang, Haibin Wu, Haoran Xu, Weijian Xu, Yifan Yang, Ziyi Yang, Donghan Yu, Ishmam Zabir, Jianwen Zhang, Li Lyna Zhang, Yunan Zhang, and Xiren Zhou. Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs. *arXiv preprint*, arXiv:2503.01743, 2025.

[48] Gemma Team. Gemma 2: Improving open language models at a practical size. *arXiv preprint*, arXiv:2408.00118, 2024.

[49] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint*, arXiv:2110.14168, 2021.

[50] Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. Orca-math: Unlocking the potential of SLMs in grade school math. *arXiv preprint*, arXiv:2402.14830, 2024.

[51] Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. OpenMathInstruct-1: A 1.8 million math instruction tuning dataset. *arXiv preprint*, arXiv:2402.10176, 2024.

[52] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.

[53] Wasi Uddin Ahmad, Aleksander Ficek, Mehrzad Samadi, Jocelyn Huang, Vahid Noroozi, Somshubra Majumdar, and Boris Ginsburg. OpenCodeInstruct: A large-scale instruction tuning dataset for code LLMs. *arXiv preprint*, arXiv:2504.04030, 2025.

[54] Zhangchen Xu, Yang Liu, Yueqin Yin, Mingyuan Zhou, and Radha Poovendran. KodCode: A diverse, challenging, and verifiable synthetic dataset for coding. *arXiv preprint*, arXiv:2503.02951, 2025.

[55] Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. TACO: Topics in algorithmic COde generation dataset. *arXiv preprint*, arXiv:2312.14852, 2023.

[56] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[57] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024.

[58] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint*, arXiv:2108.07732, 2021.

[59] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*, 2025.

[60] Songming Zhang, Xue Zhang, Tong Zhang, Bojie Hu, Yufeng Chen, and Jinan Xu. KDFlow: A user-friendly and efficient knowledge distillation framework for large language models. *arXiv preprint*, arXiv:2603.01875, 2026.

[61] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint*, arXiv:2107.03374, 2021.

[62] Keyu Li, Junhao Shi, Yang Xiao, Mohan Jiang, Jie Sun, Yunze Wu, Dayuan Fu, Shijie Xia, Xiaojie Cai, Tianze Xu, Weiye Si, Wenjie Li, Dequan Wang, and Pengfei Liu. Agencybench: Benchmarking the frontiers of autonomous agents in 1m-token real-world contexts. *arXiv preprint*, arXiv:2601.11044, 2026.

[63] Yang Xiao, Mohan Jiang, Jie Sun, Keyu Li, Jifan Lin, Yumin Zhuang, Ji Zeng, Shijie Xia, Qishuo Hua, Xuefeng Li, Xiaojie Cai, Tongyu Wang, Yue Zhang, Liming Liu, Xia Wu, Jinlong Hou, Yuan Cheng, Wenjie Li, Xiang Wang, Dequan Wang, and Pengfei Liu. Limi: Less is more for agency. *arXiv preprint*, arXiv:2509.17567, 2025.

[64] Yunze Wu, Dayuan Fu, Weiye Si, Zhen Huang, Mohan Jiang, Keyu Li, Shijie Xia, Jie Sun, Tianze Xu, Xiangkun Hu, Pengrui Lu, Xiaojie Cai, Lyumanshan Ye, Wenhong Zhu, Yang Xiao, and Pengfei Liu. Innovatorbench: Evaluating agents' ability to conduct innovative LLM research. *arXiv preprint*, arXiv:2510.27598, 2025.

[65] Ji Zeng, Dayuan Fu, Tiantian Mi, Yumin Zhuang, Yaxing Huang, Xuefeng Li, Lyumanshan Ye, Muhang Xie, Qishuo Hua, Zhen Huang, Mohan Jiang, Hanning Wang, Jifan Lin, Yang Xiao, Jie Sun, Yunze Wu, and Pengfei Liu. davinci-dev: Agent-native mid-training for software engineering. *arXiv preprint*, arXiv:2601.18418, 2026.

[66] Yukang Feng, Jianwen Sun, Zelai Yang, Jiaxin Ai, Chuanhao Li, Zizhen Li, Fanrui Zhang, Kang He, Rui Ma, Jifan Lin, Jie Sun, Yang Xiao, Sizhuo Zhou, Wenxiao Wu, Yiming Liu, Pengfei Liu, Yu Qiao, Shenglin Zhang, and Kaipeng Zhang. Longcli-bench: A preliminary benchmark and study for long-horizon agentic programming in command-line interfaces. *arXiv preprint*, arXiv:2602.14337, 2026.

[67] Dayuan Fu, Shenyu Wu, Yunze Wu, Zerui Peng, Yaxing Huang, Jie Sun, Ji Zeng, Mohan Jiang, Lin Zhang, Yukun Li, Jiarui Hu, Liming Liu, Jinlong Hou, and Pengfei Liu. davinci-env: Open swe environment synthesis at scale. *arXiv preprint*, arXiv:2603.13023, 2026.

[68] Dayuan Fu, Yunze Wu, Xiaojie Cai, Lyumanshan Ye, Shijie Xia, Zhen Huang, Weiye Si, Tianze Xu, Jie Sun, Keyu Li, Mohan Jiang, Junfei Wang, Qishuo Hua, Pengrui Lu, Xiangkun Hu, Yang Xiao, and Pengfei Liu. Argo: Asynchronous rollout with human guidance for research agent optimization. *OpenReview preprint*, 2026.

[69] Qiyong Zhong, Mao Zheng, Mingyang Song, Xin Lin, Jie Sun, Houcheng Jiang, Xiang Wang, and Junfeng Fang. Sod: Step-wise on-policy distillation for small language model agents. *arXiv preprint*, arXiv:2605.07725, 2026.

[70] Gengsheng Li, Tianyu Yang, Junfeng Fang, Mingyang Song, Mao Zheng, Haiyun Guo, Dan Zhang, Jinqiao Wang, and Tat-Seng Chua. Unifying group-relative and self-distillation policy optimization via sample routing. *arXiv preprint*, arXiv:2604.02288, 2026.

[71] Jie Sun, Junkang Wu, Jiancan Wu, Zhibo Zhu, Xingyu Lu, Jun Zhou, Lintao Ma, and Xiang Wang. Robust preference optimization via dynamic target margins. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 5399–5416, 2025.

[72] Jie Sun, Tianyu Zhang, Houcheng Jiang, Kexin Huang, Xiang Shu, Zhibo Zhu, Lintao Ma, Xingyu Lu, Jun Zhou, Junkang Wu, Chi Luo, An Zhang, Jiancan Wu, and Xiang Wang. Lampval: Large language models empower personalized valuation in auction. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 579–595, 2025.

[73] Jie Sun, Yu Liu, Lu Han, Qiwen Deng, Xiang Shu, Yang Xiao, Xingyu Lu, Jun Zhou, Pengfei Liu, Lintao Ma, Jiancan Wu, and Xiang Wang. Sepseq: A training-free framework for long numerical sequence processing in llms. *arXiv preprint*, arXiv:2604.07737, 2026.

[74] Yongduo Sui, Shuyao Wang, Jie Sun, Zhiyuan Liu, Qing Cui, Longfei Li, Jun Zhou, Xiang Wang, and Xiangnan He. A simple data augmentation for graph classification: A perspective of equivariance and invariance. *ACM Transactions on Knowledge Discovery from Data*, 19(2): 1–24, 2025.

[75] Yongduo Sui, Jie Sun, Shuyao Wang, Zemin Liu, Qing Cui, Longfei Li, and Xiang Wang. A unified invariant learning framework for graph classification. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025.

A Training Data Construction and Curation

This appendix describes the construction of the cold-start SFT corpus that produces the warm-start checkpoint shared by all OPD methods. We use public math and code datasets as prompt sources, apply filtering and deduplication, generate multiple teacher responses per prompt, and select one response according to correctness, executability, and formatting criteria. The SFT training procedure that consumes this corpus is described in Appendix B.

A.1 Composition

The corpus contains 10,000 examples: 5,800 mathematical reasoning examples and 4,200 code-generation examples. The math/code split roughly matches the relative emphasis of the two task families in our evaluation suite, and within each family we draw from multiple sources to avoid overfitting to any single dataset's style.

For mathematical reasoning, we sample 1,800 prompts from the GSM8K [49] training split, 2,200 from Orca-Math 200K [50], 1,000 from OpenMathInstruct-1 [51], and 800 from MATH [52], the last with the MATH-500 evaluation split [57] excluded. The MATH subset is drawn from levels 1–4 (150/150/350/150, respectively); we exclude level 5 because its solutions frequently exceed our SFT context budget and exhibit high teacher-side variance, both of which inject noise into the warm-start signal without proportionate benefit. Excluded difficulty levels are still represented in evaluation through MATH-500.

For code generation, we sample 1,800 prompts from OpenCodeInstruct [53], 900 from KodCode [54], 900 from TACO [55], and 600 from CodeContests [56]. The resulting subset covers function-level Python synthesis and competitive-programming I/O formats, with a preference toward examples that admit reliable execution-based verification.

A.2 Prompt Filtering and Deduplication

Before teacher generation, we filter prompts to remove noisy, duplicated, ill-formed, overly long, or difficult-to-verify instances, and we run overlap checks against held-out evaluation data using normalized text and near-duplicate matching to prevent test contamination.

For math sources, we retain text-only problems with clear statements, well-defined answer formats, and complete textual solutions. We discard examples with malformed questions, inconsistent answer formats, excessive derivation length, embedded code blocks, code-interpreter traces, or dependencies on diagrams, Asymptote figures, or other non-textual reasoning artifacts. We additionally apply problem-level near-duplicate filtering to suppress repeated template families.

For code sources, we keep only Python tasks with a well-defined solution target. When tests are available, we require compatibility with execution-based checking. We discard debugging, iterative-repair, SQL, shell, front-end, multi-file, interactive, project-dependent, and non-executable tasks. For contest-style sources (TACO, CodeContests), we further remove problems whose statements exceed our context budget or cannot be cleanly converted into standard Python generation examples.

A.3 Teacher Response Generation and Selection

For each retained prompt, the teacher samples 8 candidate responses with temperature 0.6 and top-$p = 0.95$, yielding a diverse but focused candidate set. We then select a single response per prompt as the SFT target.

Verification-based selection. When automatic verification is available, we keep only candidates that pass the corresponding checker. For math, we use `math_verify`[1] to compare extracted final answers under symbolic equivalence (handling fractions, equivalent algebraic forms, and standard formatting variations). For code, we execute each candidate against the dataset's provided unit tests in a sandboxed Python environment with a per-test wall-clock timeout of 10 seconds and a memory cap of 128 MB. When multiple candidates pass, we keep the shortest valid response after

[1] https://github.com/huggingface/math-verify

Table 2: Composition of the cold-start SFT corpus. Public datasets are used as prompt sources; supervised targets are generated by Qwen2.5-7B-Instruct [46] / Phi-4-mini-instruct [47] and selected through the response-selection pipeline described in Appendix A.

Domain	Prompt source	# Examples
Math	GSM8K train [49]	1,800
	Orca-Math 200K [50]	2,200
	OpenMathInstruct-1 [51]	1,000
	MATH, excluding MATH-500 [52, 57]	800
	Math subtotal	*5,800*
Code	OpenCodeInstruct [53]	1,800
	KodCode [54]	900
	TACO [55]	900
	CodeContests [56]	600
	Code subtotal	*4,200*
	Total	**10,000**

standardized formatting normalization, which suppresses unnecessary verbosity and limits incidental teacher-specific style.

Quality-filter selection. When verification is unavailable or partial (e.g., open-ended math without a canonical final-answer field, or code prompts without supplied tests), we apply rule-based quality filters: responses must contain a non-empty final answer in the expected format (boxed final answer for math, a complete fenced Python code block with a defined entry point for code), must not exceed 4096 tokens, and must not contain unterminated reasoning, repeated boilerplate, or empty solution bodies. The full filter rule set is documented in our released curation script. Among candidates passing all filters, we again prefer the shortest.

Role of the public datasets. Public datasets are used as *prompt sources only*; the supervised targets are generated and selected through the unified teacher-response pipeline above. This keeps warm-start supervision consistent with the teacher policy used during the subsequent OPD stage and prevents distributional drift between the two training phases. Source attribution and metadata are preserved per example throughout the pipeline.

A.4 Licenses, Attribution, and Release

Table 3 summarizes the public assets used in this work along with their roles and license terms. For each example in the released corpus we retain the source dataset, split, original identifier when available, source license, teacher model identifier, and generation configuration.

Because prompt sources carry different licenses and terms, we do not assign a single license to the constructed corpus. Per-example source attribution and license metadata are retained, and downstream users are responsible for compliance with each original asset's terms; assets with non-commercial or mixed-source terms are flagged accordingly.

For anonymous review, we release the dataset construction scripts, filtering rules, source identifiers, license metadata, checksums, and a small data sample as supplementary material. The full corpus can be reproduced from the original sources using the documented filtering and teacher-generation pipeline. Upon acceptance, we will release the finalized documentation, scripts, metadata, and all data artifacts whose redistribution is permitted in a public repository.

B Training Details

For each student model, training proceeds in two stages: a warm-start SFT stage to produce a non-trivial starting policy, followed by on-policy distillation. All OPD methods compared in this work share the same warm-start checkpoint, the same training prompts, and the same OPD hyperparameters,

Table 3: Source assets used in this work. Public datasets are used as prompt sources for constructing the cold-start SFT corpus. The final supervised targets are generated by the teacher model and selected through the response-selection pipeline described in Appendix A. We preserve source attribution and license metadata for each example.

Asset	Category	Role in This Work	License / Terms
KDFlow [60]	Code	Codebase	MIT
Qwen2.5-7B-Instruct [46]	Model	Teacher model	Apache-2.0
Gemma-2-2B-IT [48]	Model	Student model	Gemma Terms of Use
Phi-4-mini-instruct [47]	Model	Student model	MIT
GSM8K [49]	Math data	Prompt source	MIT
Orca-Math 200K [50]	Math data	Prompt source	MIT
OpenMathInstruct-1 [51]	Math data	Prompt source	NVIDIA License
MATH[†] [52]	Math data	Prompt source	MIT
OpenCodeInstruct [53]	Code data	Prompt source	CC BY 4.0
KodCode [54]	Code data	Prompt source	CC BY-NC 4.0
TACO[‡] [55]	Code data	Prompt source	Apache-2.0 / mixed permissive terms
CodeContests[§] [56]	Code data	Prompt source	Apache-2.0 / CC BY 4.0

[†] We use examples from the original MATH dataset [52] after excluding MATH-500 [57] evaluation instances. The source MATH dataset is released under the MIT license. [‡] TACO [55] is released under Apache-2.0 by its authors, while its official license note states that the dataset may include content under MIT and CC BY 4.0 terms. [§] CodeContests [56] licenses code under Apache-2.0 and non-code materials under CC BY 4.0; additional third-party source terms may apply.

and differ only in how teacher and student predictions are compared under tokenizer mismatch. Hyperparameters for both stages are summarized in Table 4; this section describes each stage in detail.

B.1 Warm-Start SFT

We first SFT each student on the 10K teacher-generated corpus described in Appendix A to obtain the warm-start checkpoint from which all OPD methods start. Sharing this initialization both gives the student a non-trivial policy before OPD [13] and ensures that differences across OPD methods are not confounded by SFT data or initial checkpoints.

We optimize with AdamW using a peak learning rate of 2×10^{-6}, a linear warmup ratio of 0.05 followed by cosine decay, and train for 2 epochs. All students share the same SFT hyperparameters (Table 4). Training is performed in bfloat16 on 8×H20 96GB.

B.2 On-Policy Distillation

Starting from the warm-start checkpoint, we train each OPD method under an identical loop: at every step the student samples a prefix under its current policy, the teacher is queried on this prefix to produce next-token supervision, and the student is updated under the method-specific loss. Training prompts, prefix-generation settings (temperature 0.6, top-$p = 0.95$), optimizer, learning-rate schedule, batch construction, and total compute budget are held identical across methods (Table 4).

For SimCT and SimpleOPD, the OPD loss is reverse KL [13, 15] applied in their respective supervision spaces—SimCT's space combines shared vocabulary tokens with minimal aligned units, while SimpleOPD restricts supervision to the exact teacher–student vocabulary overlap.

B.3 SimCT Algorithm

Algorithm 1 summarizes SimCT. Given a prompt x, the student first generates a full on-policy response y. At each prediction position t, teacher and student next-token distributions p_T^t and p_S^t are evaluated under the same prefix $(x, y_{<t})$. SimCT constructs a common supervision space $\mathcal{U}_t$ from shared vocabulary tokens augmented with minimal aligned units induced by local tokenizer-boundary

Algorithm 1 Simple Cross-Tokenizer OPD (SimCT)

Require: Teacher $\mathcal{T}$, student $\mathcal{S}_\theta$, tokenizers τ_T, τ_S, vocabularies $\mathcal{V}_T, \mathcal{V}_S$, prompts $\mathcal{X}$
Ensure: Updated student $\mathcal{S}_\theta$

for each OPD training step **do**
  Sample prompts $x \sim \mathcal{X}$
  Generate student rollouts $y \sim \mathcal{S}_\theta(\cdot \mid x)$

  Construct the SimCT supervision space.
  Tokenize the same rollout with both tokenizers:

$$\mathbf{y}^T \leftarrow \tau_T(y), \qquad \mathbf{y}^S \leftarrow \tau_S(y).$$

  Extract minimal aligned units:

$$\mathcal{A}(y) \leftarrow \mathrm{MinimalAlignedUnits}(\mathbf{y}^T, \mathbf{y}^S, y).$$

  Define the SimCT supervision space as in Eq. (6):

$$\mathcal{U}_{\mathrm{SimCT}}(y) \leftarrow (\mathcal{V}_T \cap \mathcal{V}_S) \cup \mathcal{A}(y).$$

  Distill on student-generated prefixes.
  $\mathcal{L}_{\mathrm{SimCT}} \leftarrow 0$
  for each student-generated prefix $x_{<t}$ in (x, y) **do**
    Compute teacher and student next-token distributions:

$$p_T^t \leftarrow p_T(\cdot \mid x_{<t}), \qquad p_S^t \leftarrow p_S(\cdot \mid x_{<t}).$$

    Compute continuation scores $s_M(\cdot \mid x_{<t})$ for $M \in \{T, S\}$ via Eq. (7):

$$s_M(u \mid x_{<t}) = \log p_M(u \mid x_{<t}) \text{ for } u \in \mathcal{V}_\cap;$$

$$s_M(u \mid x_{<t}) = \tfrac{1}{k} \textstyle\sum_{j=1}^{k} \log p_M(v_j \mid x_{<t}, v_{<j}) \text{ for } u \in \mathcal{A}(y).$$

    Map both models to the SimCT supervision space using Eq. (8):

$$q_T^t \leftarrow \Pi_T^{\mathrm{SimCT}}(p_T^t), \qquad q_S^t \leftarrow \Pi_S^{\mathrm{SimCT}}(p_S^t).$$

    Accumulate the SimCT loss as in Eq. (9):

$$\mathcal{L}_{\mathrm{SimCT}} \leftarrow \mathcal{L}_{\mathrm{SimCT}} + \mathcal{D}(q_S^t, q_T^t).$$

  end for
  Update θ using $\nabla_\theta \mathcal{L}_{\mathrm{SimCT}}$
end for
return $\mathcal{S}_\theta$

mismatches, and scoring operators Π_T, Π_S map both distributions into comparable distributions q_T^t, q_S^t over $\mathcal{U}_t$ on which reverse KL is applied.

C Evaluation Benchmarks and Metrics

C.1 Benchmarks

We evaluate on four benchmarks covering complementary task regimes. **GSM8K** [49] consists of grade-school word problems requiring multi-step arithmetic from natural-language descriptions; final answers are short numerical values, but solving the problems demands a coherent reasoning trajectory. **MATH-500** [52, 57] draws from competition-style problems spanning algebra, geometry, number theory, and combinatorics; answers may be symbolic forms, fractions, or equations, making the task sensitive to both reasoning quality and output normalization. **MBPP** [58] measures function-level Python synthesis from natural-language specifications, evaluated by the dataset's provided unit

Table 4: Training hyperparameters for warm-start SFT and on-policy distillation. All students share the same SFT hyperparameters; all OPD methods share the same OPD hyperparameters and differ only in supervision construction.

	Warm-Start SFT	On-Policy Distillation
Optimizer	AdamW	AdamW
Peak learning rate	2×10^{-6}	1×10^{-6}
Warmup ratio	0.05	0.05
LR schedule	Cosine decay	Cosine decay
Weight decay	0.0	0.0
Training length	2 epochs	2 epochs
Per-device batch size	2	1
Gradient accumulation	4	8
Effective batch size	64	64
Max sequence length	4096	4096
Rollout temperature	—	0.6
Rollout top-p	—	0.95
Rollout max length	—	4096
Loss (SimCT, SimpleOPD)	—	Reverse KL
Precision	bfloat16	bfloat16
Hardware	8× H20	8× H20

tests. **LiveCodeBench-v6** [59] evaluates code generation on contemporary problems with reduced contamination, requiring more robust algorithmic reasoning under hidden tests.

These benchmarks expose tokenizer mismatch in different ways—math evaluation involves symbolic expressions and rationale tokens, while code involves identifiers, indentation, and syntax fragments—so they jointly test whether SimCT recovers useful supervision across heterogeneous output structures rather than overfitting to a single format.

C.2 Decoding

All benchmarks are evaluated zero-shot: each model receives only the task prompt, with no in-context demonstrations. This isolates capability learned during SFT and OPD from prompt-level effects, and avoids tokenizer-dependent confounds introduced by few-shot examples.

For each benchmark, we use a fixed prompt template across all methods and student models. Math prompts ask the model to solve the problem and place the final answer in `\boxed{}`; code prompts present the specification and ask for a Python solution in the benchmark's expected format. Full prompt templates are released in our supplementary code.

We decode with nucleus sampling at temperature 0.6 and top-$p = 0.95$, sampling one completion per instance and repeating evaluation for 5 independent runs with different random seeds. The maximum generation length is fixed per benchmark and held identical across methods (4096 tokens for GSM8K, MATH-500 and LiveCodeBench-v6, and 2048 tokens for MBPP).

C.3 Scoring

We report pass@1 [61] averaged over the 5 runs; per-run mean±std is in Table 1.

For math, we extract the final answer (preferring content inside `\boxed{}` when present) and compare against the reference using `math_verify`[2] under symbolic equivalence; this is the same checker used during SFT data curation (Appendix A), ensuring the verification standard is consistent across stages. For code, generated solutions are executed in a sandboxed Python environment with a per-test timeout of 10 seconds and a memory cap of 128 MB; we use the official MBPP test harness

[2] https://github.com/huggingface/math-verify

and the LiveCodeBench-v6 official evaluator. Postprocessing rules, the answer-extraction regex, and the sandbox configuration are identical across all compared methods.

D Theory and Proofs

This appendix provides the formal definitions and proofs for Theorem 1 and Proposition 1. The section has two goals. First, we formalize why SimCT aligned units are the minimal text units that remain jointly tokenizable by the teacher and student tokenizers. Second, we prove the property used in the supervision-space ablation: merging minimal aligned units removes exactly the within-span KL signal that distinguishes teacher and student preferences inside each merged span.

D.1 Tokenizer-Induced Spans and Minimal Aligned Units

Let y be a finite text span represented over a fixed atomic string representation, such as characters or bytes. We identify y with the interval $[0, |y|]$. The teacher tokenizer induces a partition

$$\mathcal{P}_T(y) = \{[b_{i-1}^T, b_i^T) : i = 1, \dots, m\}, \qquad 0 = b_0^T < \cdots < b_m^T = |y|,$$

and the student tokenizer induces a partition

$$\mathcal{P}_S(y) = \{[b_{j-1}^S, b_j^S) : j = 1, \dots, n\}, \qquad 0 = b_0^S < \cdots < b_n^S = |y|.$$

Each interval in $\mathcal{P}_T(y)$ is a complete teacher-token span, and each interval in $\mathcal{P}_S(y)$ is a complete student-token span.

We first take the union of teacher and student token boundaries:

$$\mathcal{B}_\cup(y) = \mathcal{B}_T(y) \cup \mathcal{B}_S(y),$$

where $\mathcal{B}_T(y) = \{b_i^T\}_{i=0}^m$ and $\mathcal{B}_S(y) = \{b_j^S\}_{j=0}^n$. Let

$$0 = c_0 < c_1 < \cdots < c_\ell = |y|$$

be the sorted elements of $\mathcal{B}_\cup(y)$. This gives atomic fragments

$$\mathcal{A}(y) = \{[c_{k-1}, c_k) : k = 1, \dots, \ell\}.$$

These fragments are only used to define the construction. A fragment may lie inside a teacher token or a student token, and therefore is not necessarily a valid supervision unit by itself.

To obtain jointly-tokenizable units, we build an undirected graph G_y whose vertices are the atomic fragments in $\mathcal{A}(y)$. Two vertices are connected if their fragments lie inside the same teacher-token span or inside the same student-token span. The connected components of G_y induce a partition of y, denoted

$$\mathcal{P}_{\mathrm{SimCT}}(y) = \{u_1, \dots, u_r\}.$$

We call each u_k a *minimal aligned unit*. Intuitively, the connected component closes all tokenizer overlaps: if splitting a span would cut through a complete teacher token or a complete student token, the corresponding fragments must remain in the same aligned unit.

D.2 Proof of Theorem 1

Proof. We prove joint tokenizability and minimality.

Joint tokenizability. Consider any unit $u \in \mathcal{P}_{\mathrm{SimCT}}(y)$. By construction, u is a connected component of G_y. If a teacher-token span intersects u, then all atomic fragments inside that teacher token are connected in G_y, so the entire teacher-token span must lie in the same connected component. Therefore, no boundary of u cuts through a teacher token, and u is a union of consecutive complete teacher tokens.

The same argument applies to the student tokenizer. If a student-token span intersects u, all atomic fragments inside that student token are connected, so the complete student-token span lies inside u. Hence u is also a union of consecutive complete student tokens. Thus u can be represented as one or more teacher tokens and also as one or more student tokens.

Minimality. Assume for contradiction that a unit u admits a strictly finer non-empty decomposition

$$u = u' \circ u'',$$

where both u' and u'' are jointly tokenizable. Let z be the boundary between u' and u''. Since both sides are unions of complete teacher tokens, z must be a teacher-token boundary. Since both sides are also unions of complete student tokens, z must be a student-token boundary. Therefore, no teacher token or student token crosses z.

It follows that no edge in G_y can connect an atomic fragment in u' to an atomic fragment in u'', because such an edge would require the two fragments to lie inside the same teacher-token span or the same student-token span, contradicting that no token crosses z. Thus u' and u'' would belong to different connected components of G_y, contradicting that u is one connected component. Therefore, no strictly finer non-empty jointly-tokenizable decomposition exists. □

Implication. The theorem shows that SimCT does not rely on an arbitrary span length. It selects the finest units that are valid under both tokenizers. Finer atomic fragments may cut through complete tokens, while coarser mismatch spans merge multiple valid aligned units.

D.3 Proof of Proposition 1: Coarsening and Removed KL Signal

We prove Proposition 1, which is used in the supervision-space ablation. For completeness, we restate the proposition before giving the proof. Let $\mathcal{U}_{\min}$ denote the minimal aligned-unit space for a student-generated prefix h. Let

$$q_T^{\min}(\cdot \mid h), \qquad q_S^{\min}(\cdot \mid h)$$

be the teacher and student distributions over $\mathcal{U}_{\min}$.

A coarsening $\mathcal{C}$ partitions $\mathcal{U}_{\min}$ into disjoint coarse spans. For each coarse span $c \in \mathcal{C}$, the coarsened distributions are

$$q_T^{\mathcal{C}}(c \mid h) = \sum_{u \in c} q_T^{\min}(u \mid h), \qquad q_S^{\mathcal{C}}(c \mid h) = \sum_{u \in c} q_S^{\min}(u \mid h).$$

This is exactly the aggregation used in Eq. 11.

Proposition 1 (restated). For any coarsening $\mathcal{C}$ of the minimal aligned-unit space,

$$\mathrm{KL}\big(q_S^{\min}(\cdot \mid h) \,\|\, q_T^{\min}(\cdot \mid h)\big) \geq \mathrm{KL}\big(q_S^{\mathcal{C}}(\cdot \mid h) \,\|\, q_T^{\mathcal{C}}(\cdot \mid h)\big)\,.$$

Moreover, the difference equals the expected within-coarse-span KL:

$$\begin{aligned}&\mathrm{KL}\big(q_S^{\min}(\cdot \mid h) \,\|\, q_T^{\min}(\cdot \mid h)\big) - \mathrm{KL}\big(q_S^{\mathcal{C}}(\cdot \mid h) \,\|\, q_T^{\mathcal{C}}(\cdot \mid h)\big) \\ &\quad = \sum_{c \in \mathcal{C}} q_S^{\mathcal{C}}(c \mid h)\, \mathrm{KL}\big(q_S^{\min}(\cdot \mid h, c) \,\|\, q_T^{\min}(\cdot \mid h, c)\big)\,,\end{aligned}$$

where

$$q_S^{\min}(u \mid h, c) = \frac{q_S^{\min}(u \mid h)}{q_S^{\mathcal{C}}(c \mid h)}, \qquad q_T^{\min}(u \mid h, c) = \frac{q_T^{\min}(u \mid h)}{q_T^{\mathcal{C}}(c \mid h)}$$

for $u \in c$, with zero-mass coarse spans omitted. Thus, coarsening removes exactly the KL signal corresponding to teacher–student discrepancies among minimal aligned units inside each merged span.

Proof. For readability, omit the conditioning on h. Let $q_S = q_S^{\min}$ and $q_T = q_T^{\min}$. For each coarse span $c \in \mathcal{C}$, write

$$Q_S(c) = \sum_{u \in c} q_S(u), \qquad Q_T(c) = \sum_{u \in c} q_T(u).$$

Then $Q_S = q_S^{\mathcal{C}}$ and $Q_T = q_T^{\mathcal{C}}$.

We decompose the minimal-unit KL by grouping terms according to their coarse span:

$$\mathrm{KL}(q_S \| q_T) = \sum_{c \in \mathcal{C}} \sum_{u \in c} q_S(u) \log \frac{q_S(u)}{q_T(u)}.$$

Table 5: **Training time comparison.** We report wall-clock training time on a single machine with 8 NVIDIA H20 GPUs. All methods use the same training budget and protocol.

Method	Qwen2.5→Gemma2	Phi4→Gemma2	Qwen2.5→Phi4	Average
SimpleOPD	2.48h	2.61h	2.50h	2.53h
SimCT	2.70h	2.73h	2.64h	2.69h
ALM	2.84h	2.88h	2.79h	2.84h
GOLD	2.93h	2.89h	2.97h	2.93h
DSKD	2.90h	2.96h	2.91h	2.92h

For any $u \in c$, write

$$q_S(u) = Q_S(c)\, q_S(u \mid c), \qquad q_T(u) = Q_T(c)\, q_T(u \mid c),$$

where

$$q_S(u \mid c) = \frac{q_S(u)}{Q_S(c)}, \qquad q_T(u \mid c) = \frac{q_T(u)}{Q_T(c)}.$$

Substituting this factorization gives

$$\begin{aligned}
\mathrm{KL}(q_S \| q_T) &= \sum_{c \in \mathcal{C}} \sum_{u \in c} Q_S(c) q_S(u \mid c) \log \frac{Q_S(c) q_S(u \mid c)}{Q_T(c) q_T(u \mid c)} \\
&= \sum_{c \in \mathcal{C}} Q_S(c) \log \frac{Q_S(c)}{Q_T(c)} + \sum_{c \in \mathcal{C}} Q_S(c) \sum_{u \in c} q_S(u \mid c) \log \frac{q_S(u \mid c)}{q_T(u \mid c)} \\
&= \mathrm{KL}(Q_S \| Q_T) + \sum_{c \in \mathcal{C}} Q_S(c)\, \mathrm{KL}(q_S(\cdot \mid c) \,\|\, q_T(\cdot \mid c)) \,.
\end{aligned}$$

Since the second term is a nonnegative weighted sum of KL divergences,

$$\mathrm{KL}(q_S \| q_T) \geq \mathrm{KL}(Q_S \| Q_T).$$

Rearranging the equality above gives the stated expression for the removed KL signal. □

Connection to the ablation. Equation 12 averages the difference above over student-generated prefixes:

$$\Delta_{\mathcal{C}} = \mathbb{E}_h \left[\mathrm{KL}(q_S^{\min}(\cdot \mid h) \| q_T^{\min}(\cdot \mid h)) - \mathrm{KL}(q_S^{\mathcal{C}}(\cdot \mid h) \| q_T^{\mathcal{C}}(\cdot \mid h)) \right].$$

By the proposition, $\Delta_{\mathcal{C}}$ is nonnegative and measures the expected within-span KL signal erased by the coarsening $\mathcal{C}$. The right panel of Figure 4 therefore directly tests the prediction suggested by the theory: coarsenings that remove more within-span teacher–student discrepancy provide weaker supervision and lead to smaller downstream gains.

D.4 Discussion

The theorem and proposition play complementary roles. The theorem identifies the finest jointly-tokenizable units available under the two tokenizer partitions. The proposition shows what is lost when those units are merged: coarsening preserves the total probability of each merged span, but discards the conditional teacher–student differences among the minimal units inside it.

These results rely only on tokenizer boundaries and probability aggregation. They do not assume a learned semantic alignment, an embedding similarity metric, or an optimal-transport plan. This matches the design of SimCT: the method changes the supervision space to the finest comparable text units while keeping the OPD objective itself unchanged.

E Computational Resources

All experiments were conducted on a single server equipped with dual AMD EPYC 9K84 CPUs, 384 logical CPU cores, 2.2TB system memory, and 8× NVIDIA H20 GPUs with approximately 96GB memory per GPU. Table 5 reports the measured wall-clock time of a single cross-tokenizer OPD run

Table 6: Estimated compute resources for reproducing the reported experiments. All runs use a single machine with 8× NVIDIA H20 96GB GPUs. Each configuration is repeated over 5 random seeds. Cost is reported in days on the 8×H20 server, computed as (number of runs × wall-clock hours per run) / 24.

Stage	Component	Runs	Runtime per run	8×H20 days
Cold start	Qwen2.5 response generation	1×5	1 h	0.21
	Phi-4 response generation	1×5	1 h	0.21
	Phi-4 SFT (Qwen-generated data)	1×5	1 h	0.21
	Gemma SFT (Qwen-generated data)	1×5	1 h	0.21
	Gemma SFT (Phi-generated data)	1×5	1 h	0.21
	Cold-start subtotal	25	—	*1.04*
Training	Qwen → Phi, five methods	5×5	3 h	3.13
	Qwen → Gemma, five methods	5×5	3 h	3.13
	Phi → Gemma, five methods	5×5	3 h	3.13
	Training subtotal	75	—	*9.38*
Evaluation	Base models (Qwen, Phi, Gemma)	3×5	1 h	0.63
	Warm-start (3 pairs)	3×5	1 h	0.63
	Qwen → Phi (5 methods)	5×5	1 h	1.04
	Qwen → Gemma (5 methods)	5×5	1 h	1.04
	Phi → Gemma (5 methods)	5×5	1 h	1.04
	Evaluation subtotal	105	—	*4.38*
			Total	**14.79**

under this hardware setting. Across teacher–student pairs and methods, one training run typically takes about 2.5–3 hours. SimCT adds only a small overhead over SimpleOPD, while being faster than ALM, GOLD, and DSKD across the reported settings.

Table 6 summarizes the compute required to reproduce the reported main experiments. To reduce variance, every stage of the pipeline (warm-start SFT, cross-tokenizer OPD training, and evaluation) is repeated 5 times with different random seeds, and the reported results in Table 1 are averaged across these runs. For the aggregate estimate, we use a conservative rounded runtime of approximately 3 hours per cross-tokenizer OPD run, consistent with the measured runtimes in Table 5. The total cost is approximately 14.8 days on a single 8× H20 server. The total consists of three parts. First, the warm-start stage costs about 1.0 day on a single 8× H20 server, covering two teacher response-generation runs and three student SFT runs (Phi-4 on Qwen-generated data, Gemma-2 on Qwen-generated data, and Gemma-2 on Phi-generated data), each repeated over 5 random seeds. Second, cross-tokenizer OPD training costs about 9.4 days on a single 8× H20 server, covering three teacher–student pairs (Qwen2.5→Phi4, Qwen2.5→Gemma-2, and Phi4→Gemma-2), each trained with five methods (SimpleOPD, SimCT, ALM, GOLD, and DSKD), and each method-pair configuration repeated over 5 random seeds, yielding $3 \times 5 \times 5 = 75$ training runs. Third, evaluation costs about 4.4 days on a single 8× H20 server, covering three base models, three warm-start checkpoints, and fifteen trained checkpoints, each evaluated over 5 independent runs.

The reported total only accounts for the runs needed to reproduce the main experimental results. It excludes preliminary studies, hyperparameter exploration, debugging runs, ablation studies in Section 4.3 and Section 4.4, and failed or repeated jobs during development. The full project compute is therefore larger than the reproducibility cost reported here.

Finally, the end-to-end wall-clock cost of cross-tokenizer OPD is dominated by student rollouts and teacher forward passes. SimCT operates at the distillation-interface level. It introduces no trainable projection module, requires no global vocabulary-level alignment, and keeps the standard OPD training pipeline unchanged. As a result, SimCT remains close to the lightweight SimpleOPD baseline and is faster than more complex cross-tokenizer baselines, while recovering substantially more teacher supervision than SimpleOPD.

F Case Study

We provide qualitative case studies on math and code examples to illustrate how different supervision interfaces can affect cross-tokenizer OPD behavior at the local level. For each example, we compare SimCT with representative baselines under the same teacher–student setting and report the final answer or execution result. These cases highlight representative local errors—an incorrect modular computation, a missing symmetry factor, a mismatched input-output convention, or an invalid function signature—that are easy to miss in aggregate scores. Several baselines recover the high-level solution template but fail on these local distinctions, which is consistent with the supervision loss caused by tokenizer mismatch as measured in Section 4.3. SimCT avoids these specific errors in the shown examples, providing qualitative intuition for how finer-grained supervision can help preserve local reasoning accuracy. We emphasize that these are *illustrative examples*, not causal evidence; the downstream gains are established quantitatively in Table 1.

Case Study 1: Number Theory (Level 4) — Qwen2.5 → Gemma-2-2B *MATH500*

Question: Find the modulo 7 remainder of the sum $1 + 3 + 5 + 7 + 9 + \cdots + 195 + 197 + 199$.
Ground-truth Answer: 4

Method	Response (Abbreviated)	Result
SimCT (Ours)	*The sequence* $1, 3, 5, \ldots, 199$ *has* $n = 100$ *terms. Sum* $= \frac{100}{2}(1 + 199) = 10000$. *Since* $10000 = 7 \times 1428 + 4$, *the remainder is* 4. **Answer:** $\boxed{4}$	✓ **Correct**
SimpleOPD	*Correctly identifies 100 terms and computes the sum as 10000. However, makes an arithmetic error in the final modular division step, computing* $10000 \div 7 = 1428$ *remainder* 3 *instead of* 4. **Answer:** 3	✗ **Incorrect**
ALM	*Incorrectly counts the number of terms as 199 (confusing the last term with the count). Computes a wrong sum and obtains remainder* 1. **Answer:** 1	✗ **Incorrect**
DSKD	*Correctly sets up the arithmetic sequence formula but makes an error in computing* $10000 \bmod 7$, *claiming* $10000 = 7 \times 1429 - 3$ *and reporting remainder* 2. **Answer:** 2	✗ **Incorrect**
GOLD	*Identifies the correct sum of 10000 but then incorrectly applies modular arithmetic, computing* $10000 \bmod 7 = 3$ *(same error as SimpleOPD).* **Answer:** 3	✗ **Incorrect**

Analysis

All methods correctly identify the arithmetic sequence structure, but diverge at the final modular arithmetic step. SimCT precisely computes $10000 = 7 \times 1428 + 4$, while baselines make arithmetic errors in the division ($7 \times 1428 = 9996$, so the remainder is $10000 - 9996 = 4$). This demonstrates that span-level contrastive training preserves the teacher's computational precision in multi-step arithmetic, whereas token-level methods accumulate small errors in the final calculation step.

Case Study 2: Counting & Probability (Level 4) — Qwen2.5 → Phi4-mini *MATH500*

Question: In how many ways can 8 people be seated around a square table with 2 people on a side? (Two configurations are considered equivalent if one is a rotation of another.)
Ground-truth Answer: 10080

Method	Response (Abbreviated)	Result
SimCT (Ours)	*Total linear arrangements:* $8! = 40320$. *A square table has 4 rotational symmetries (0°, 90°, 180°, 270°). Dividing:* $8!/4 = 40320/4 = 10080$. **Answer:** $\boxed{10080}$	✓ **Correct**
SimpleOPD	*Attempts Burnside's lemma but incorrectly computes* $\mathrm{Fix}(180°) = 4! \times 2^4 = 384$ *instead of 0, yielding* $\frac{40320+384}{4} = 10176$. **Answer:** 10176	✗ **Incorrect**
ALM	*Incorrectly treats the problem as a circular permutation first* $(8!/8 = 7!)$*, then divides by 4 again for square symmetry, double-counting:* $7!/4 = 1260$. **Answer:** 1260	✗ **Incorrect**
DSKD	*Fixes one person at a corner, then incorrectly partitions the remaining 7 people into "pairs" using* $\frac{7!}{(2!)^3} = 630$*, conflating seating with pairing.* **Answer:** 630	✗ **Incorrect**
GOLD	*Fixes one person, computes* $7!$*, then multiplies by* $(2!)^4 = 16$ *(incorrectly adding intra-side permutations already counted), yielding* $\frac{7!\times 16}{4} = 20160$. **Answer:** 20160	✗ **Incorrect**

Analysis

SimCT applies the clean and correct approach: total permutations divided by rotational symmetries $(8!/4)$. Each baseline attempts a different—and incorrect—strategy: SimpleOPD misapplies Burnside's lemma; ALM double-divides by both circular and square symmetries; DSKD confuses seating with pairing; and GOLD redundantly multiplies by intra-side arrangements. The diversity of errors highlights that this problem requires precise understanding of symmetry groups—a capability that span-level contrastive training successfully transfers from the teacher.

Case Study 3: Code Generation — Qwen2.5 → Gemma-2-2B *LiveCodeBench*

Problem (abc389_b): You are given an integer X not less than 2. Find the positive integer N such that $N! = X$. It is guaranteed that there is exactly one such N.
Constraints: $2 \leq X \leq 3 \times 10^{18}$

Method	Generated Code (Key Logic)	Tests	Result

SimCT (Ours)	`def findN(x):`
	`  n = 1`
	`  fact = 1`
	`  while fact < x:`
	`  n += 1`
	`  fact *= n`
	`  return n`
	`x = int(input().strip())`
	`print(findN(x))`
		42/42	✓ Pass

SimpleOPD	`def find_n():`
	`  for i in range(2, int(1e18) + 1):`
	`  if factorial(i) == int(input()):`
	`  return i`
	▷ Calls `input()` inside loop; iterates up to 10^{18} (TLE)
		3/42	✗ Fail

ALM	`def find_N():`
	`  X = int(input())`
	`  N = 1`
	`  while True:`
	`  N = factorial(N)  # overwrites N!`
	`  if N == X: return N`
	`  else: N += 1`
	▷ Overwrites N with $N!$; reads input twice; infinite loop
		0/42	✗ Fail

DSKD	`# Outputs explanation text before code`
	`def find_n():`
	`  X = int(input())`
	`  for n in range(2, X + 1):`
	`  if factorial(n) == X: return n`
	▷ Reads input twice; loops up to X with recursive factorial (TLE/stack overflow)
		0/42	✗ Fail

GOLD	`def find_n():`
	`  for n in range(2, 3*10**18 + 1):`
	`  if factorial(n) == int(input()):`
	`  return n`
	▷ Calls `input()` inside loop; recursive factorial causes stack overflow
		3/42	✗ Fail

Analysis

SimCT generates an efficient $O(\log X)$ iterative solution: incrementally multiply until reaching X. All baselines share critical flaws: (1) calling `input()` inside the search loop instead of reading it once upfront, and (2) using brute-force iteration up to X or 10^{18} with expensive factorial recomputation, causing timeout or stack overflow on large inputs. This demonstrates that span-level contrastive training transfers not only syntactic correctness but also algorithmic efficiency and proper I/O handling from the teacher model.

Case Study 4: Code Generation — Qwen2.5 → Phi4-mini *MBPP*

Problem: Write a python function to find the largest number that can be formed with the given digits.
Test Cases:

```
assert find_Max_Num([1,2,3], 3) == 321
assert find_Max_Num([4,5,6,1], 4) == 6541
assert find_Max_Num([1,2,3,9], 4) == 9321
```

Method	Generated Code (Key Logic)	Result

SimCT (Ours)	`def find_Max_Num(digits, n):`
	`  digits.sort(reverse=True)`
	`  return int(''.join(map(str, digits[:n])))`
	▷ Correct signature `(digits, n)` matching test cases
		✓ Pass

SimpleOPD	`def find_Max_Num(digits):`
	`  sorted_digits = sorted(digits, reverse=True)`
	`  return int(''.join(map(str, sorted_digits)))`
	▷ Missing parameter `n`; call `find_Max_Num([1,2,3], 3)` raises `TypeError`
		✗ Fail

ALM	`def find_Max_Num(digits):`
	`  digits.sort(reverse=True)`
	`  return int(''.join(map(str, digits)))`
	▷ Missing parameter `n`; same `TypeError` on test invocation
		✗ Fail

DSKD	`def find_Max_Num(digits):`
	`  return int(''.join(sorted(map(str,digits), reverse=True)))`
	▷ Missing parameter `n`; additionally sorts digit *strings* (wrong for multi-digit)
		✗ Fail

GOLD	`def find_Max_Num(digits):`
	`  digits.sort(reverse=True)`
	`  return int(''.join(map(str, digits)))`
	▷ Missing parameter `n`; identical failure mode as ALM
		✗ Fail

Analysis

This example reveals a subtle but critical failure: all baseline methods generate *functionally correct* sorting logic, but define the function with only one parameter `(digits)` instead of the required two-parameter signature `(digits, n)`. When the test harness calls `find_Max_Num([1,2,3], 3)`, all baselines raise a `TypeError` due to the unexpected second argument. SimCT correctly infers the expected function interface from the problem context, demonstrating that span-level contrastive training better preserves the teacher's ability to understand implicit API contracts and generate code with correct function signatures.

G Broader Scope of Supervision Interface Design

SimCT focuses on cross-tokenizer on-policy distillation, where the central difficulty is that teacher and student predictions are not directly comparable in their original token spaces. Although our experiments are conducted on math and code generation, this issue is part of a broader challenge: as LLMs are trained and evaluated in longer and more interactive settings, the form in which supervision is represented can strongly affect what learning signal is preserved.

Recent work has increasingly studied LLMs beyond isolated response generation, including autonomous agents, data-efficient agency training, research-oriented agents, software-engineering agents, command-line programming, environment synthesis, and human-guided rollout optimization [62, 63, 64, 65, 66, 67, 68]. These settings often contain mixed textual structures such as natural language instructions, code edits, shell commands, tool outputs, environment states, and intermediate

reasoning traces. They therefore provide a natural testbed for studying whether teacher feedback remains comparable and informative when student trajectories become longer, more structured, and more tool-dependent. Step-wise on-policy distillation for small language model agents further shows that OPD-style supervision can be adapted to such agentic trajectories [69]. From this perspective, SimCT addresses a complementary question: when OPD is performed across models with heterogeneous tokenizers, how should the teacher and student distributions be expressed so that useful feedback is not discarded before the objective is even applied?

This supervision-interface view is also related to recent work on how on-policy or preference-based signals should be selected, routed, or calibrated. Analyses of OPD failure modes emphasize that teacher feedback on student-generated prefixes can be unstable or unevenly useful [38], while sample-routing formulations connect group-relative policy optimization and self-distillation under a unified view of policy optimization [70]. Preference-oriented LLM training provides another example: dynamic target margins adapt the supervision margin for robust preference optimization [71], and personalized valuation studies how LLMs can model user-dependent value in auction scenarios [72]. These works change the source, strength, or routing of supervision, whereas SimCT changes the space in which teacher and student predictions are compared. The two directions are orthogonal and could be combined in future OPD systems.

Finally, the need for an appropriate comparison interface is not limited to distillation. Long numerical sequence processing shows that LLM behavior can be sensitive to how structured sequences are formatted and segmented [73]. In graph learning, equivariance and invariance have been used to construct representations that preserve task-relevant information while reducing irrelevant transformation effects [74, 75]. SimCT follows the same broad principle in a different setting: rather than comparing teacher and student models in tokenizer-specific spaces, it constructs a common supervision space over aligned text units so that the distillation signal is expressed at a level both models can realize.